Safe Adaptive Ensemble Reinforcement Learning (SAERL) for Physics-Consistent Fast Charging of Lithium-Ion Battery Packs

Robin K. E. Tau[1,*], Abid Yahya[1], Mmoloki Mangwala[1] ,Nonofo M. J. Ditshego[1],

[1]Department of Electrical and Communications Systems, Botswana International University of Science and Technology, Palapye, Botswana

*Correspondence: tr1800236@studentmail.biust.ac.bw

February 17, 2026

Abstract

Fast charging of lithium-ion battery packs remains a critical challenge for electric vehicle adoption, requiring careful balance between charge speed, thermal safety, and lifetime degradation. Existing approaches—rule-based constant-current constant-voltage (CCCV), model predictive control (MPC), and pure reinforcement learning—struggle to simultaneously adapt to cell heterogeneity, enforce hard safety constraints, and maintain performance under sensor noise. This paper introduces SAERL (Safe Adaptive Ensemble Reinforcement Learning), a hybrid framework that combines three complementary predictors (GRU, MLP, random forest) with an adaptive gating mechanism that fuses predictions based on instantaneous error and uncertainty. A residual policy, warm-started via behavioral cloning and fine-tuned with PPO, outputs corrections to an MPC baseline, while a differentiable safety shield enforces voltage, temperature, and SoC limits with a guaranteed fallback action. We evaluate SAERL on 15 diverse fast-charging scenarios spanning three public datasets (NASA, CALCE, MATR) using a high-fidelity electro-thermal-aging pack model with explicit cell-to-cell variance. SAERL achieves a 40% acceptance rate (6/15 scenarios), compared to 0% for both CCCV and MPC, while maintaining zero safety events in 12/15 runs. Against MPC, SAERL reduces time-to-80% SoC by 7.7 min (8.6%) and improves final SoC by 0.16%; against CCCV, it cuts safety violations by 97.3%. Inference latency (26.3 ms) is well within BMS real-time constraints. Ablation studies confirm that the safety shield is the most critical component, and that family-specific fine-tuning further boosts performance. The code, datasets, and trained models are released to support reproducible research.

Keywords: reinforcement learning, fast charging, lithium-ion batteries, uncertainty quantification, ensemble learning, battery management systems

1 Introduction

The global transition to electric mobility hinges on battery systems that can be recharged as quickly and safely as refueling a conventional vehicle. Lithium-ion batteries, the dominant energy storage technology, impose strict operational limits: exceeding voltage, temperature, or current boundaries accelerates capacity fade and can trigger catastrophic thermal runaway [**?** **?**]. Fast charging thus becomes a constrained optimization problem—minimize charging time while satisfying safety constraints and minimizing degradation.

Industrial practice overwhelmingly relies on the constant-current constant-voltage (CCCV) protocol, which is simple and robust but inherently conservative, leaving significant time on the table [**?**]. Model predictive control (MPC) can explicitly handle constraints and optimize a finite horizon, yet its performance degrades when model mismatch or cell-to-cell variations are present, and its computational cost scales unfavorably with pack size [**?**]. Reinforcement learning (RL) offers an appealing alternative: a policy can learn to exploit nonlinear dynamics and adapt online. However, pure model-free RL methods often violate constraints during exploration and require prohibitively many interactions with the physical system [**?**].

To overcome these limitations, we propose **SAERL (Safe Adaptive Ensemble Reinforcement Learning)** a hybrid framework that tightly integrates three complementary predictors (a quantile GRU, a quantile MLP, and a random forest with tree-variance uncertainty) with an adaptive gating network, a residual policy trained via offline-to-online RL, and a differentiable safety shield. SAERL operates directly on a battery pack containing cells with inherent parameter variance, using only commonly available measurements (voltage, current, surface temperature).

The remainder of this paper is organized as follows. Section 2 reviews related work on charging optimization and RL for battery control. Section 3 formulates fast charging as a constrained Markov decision process. Section 4 details the SAERL architecture, including the ensemble, adaptive gate, residual policy, and safety shield. Section 5 describes the battery and pack models, datasets, training pipeline, and evaluation protocol. Section 6 presents main results, robustness analysis, and ablation studies. Section 7 interprets the findings, and Section 8 discusses limitations and future directions. We conclude in Section 9. All code, data, and trained models are publicly available to foster reproducible research.

2 Background and Related Work

2.1 Charging Strategies

The Constant Current Constant Voltage (CCCV) protocol is the de facto standard charging strategy for lithium-ion batteries, prized for its simplicity, reliability, and straightforward implementation [**?**].

CCCV charging proceeds in two distinct phases. In the first, *constant current* (CC) phase, a fixed current—typically specified as a C-rate—is applied until the battery terminal voltage reaches

a predefined upper cutoff limit. The current is then held constant while the charger switches to the *constant voltage* (CV) phase. During this phase, the current decays exponentially as the cell approaches full charge, and the process terminates when the current falls below a set threshold (e.g., C/20) [**?**].

This dual-mode structure intrinsically protects the cell: the CC phase limits current-related stress, while the CV phase prevents overvoltage. Because the protocol requires no knowledge of internal battery states such as state of charge (SoC) or state of health (SoH), it is easily implemented in low-cost chargers and remains highly robust across different cell formats and chemistries.

The dominance of CCCV stems from several practical advantages. It is completely open-loop and does not rely on a battery model, eliminating the need for parameter identification or online estimation. Its control law is trivial—saturate voltage, then regulate voltage—making it suitable for analog and low-end digital controllers. As noted in [**?**], this lack of parameterization ensures consistent behaviour across a wide range of operating conditions, albeit at the cost of optimality.

Despite its widespread use, CCCV exhibits significant drawbacks, particularly for fast charging applications. The most critical limitation is the prolonged duration of the CV phase when high CC rates are employed. A high charging current causes the terminal voltage to hit the cutoff limit early, leaving the cell partially charged; the subsequent CV phase must then deliver the remaining capacity at decaying current, which is inherently time-inefficient. This behaviour leads to diminishing returns: doubling the CC rate rarely halves the total charge time.

Moreover, the fixed voltage threshold used in the CV phase is chosen for a fresh cell at room temperature. As the battery ages, its internal resistance rises and its open-circuit voltage (OCV) characteristics shift. A static voltage limit may therefore cause undercharging (reduced usable capacity) or, more dangerously, overcharging and lithium plating when the cell is cold or aged [**?**]. Standard CCCV cannot adapt to such changes, creating a critical safety–performance trade-off [**?**].

Model predictive control (MPC) has been extensively studied for charging optimization. By solving a finite-horizon optimal control problem at each step, MPC can explicitly enforce constraints on voltage, temperature, and state of charge (SoC). Zou et al. [**?**] demonstrated MPC for a single-cell electro-thermal model, while Yan et al. [**?**] extended it to battery packs. However, MPC relies on an accurate parametric model; when the model is mismatched or when cell parameters vary, constraint satisfaction is not guaranteed and performance degrades.

Model Predictive Control (MPC) has garnered significant attention as an optimisation-based strategy for lithium-ion battery fast charging. Unlike conventional rule-based protocols such as CCCV, MPC employs a dynamic model of the battery to predict future system behaviour over a finite horizon and computes an optimal current profile by solving a constrained optimisation problem at each sampling instant. This receding-horizon approach enables the controller to anticipate future constraint violations, explicitly manage trade-offs between competing objectives, and incorporate multi-physical dynamics including electrical, thermal, and aging phenomena. Early work demonstrated the feasibility of MPC for single-cell electro-thermal control [**?**], while subsequent research extended the framework to battery packs, addressing cell-to-cell variations and balancing

constraints [?]. More recent contributions have refined both the modelling fidelity and the optimisation methodologies [? ? ?], positioning MPC as a cornerstone of next-generation intelligent battery management systems.

The principal advantage of MPC lies in its ability to explicitly enforce hard constraints on critical battery variables such as terminal voltage, current, state of charge, and temperature. By embedding these constraints directly into the optimisation problem, MPC can prevent operation in hazardous regions associated with accelerated degradation or safety risks, such as lithium plating under low-temperature, high-rate conditions [?]. This constraint-handling capability is inherent to the formulation and does not require heuristic tuning of limit margins.

Furthermore, MPC provides a systematic framework for multi-objective optimisation. Charging time, energy efficiency, and capacity fade can be weighted within a single cost function, allowing the controller to navigate the inherent trade-offs between fast charging and battery longevity. A nonlinear MPC formulation that incorporated an electro-thermal-aging coupled model achieved a 9.8% reduction in charging time while simultaneously limiting capacity fade [?]. The predictive nature of MPC also enables feedforward action; by forecasting future voltage or temperature trajectories, the controller can preemptively reduce current before a constraint is violated, yielding smoother and safer transients than purely reactive feedback schemes.

Despite its theoretical and practical strengths, MPC faces several barriers that have impeded its widespread adoption in commercial battery management systems.

The performance of MPC is contingent upon the accuracy of the embedded predictive model. Equivalent circuit models, while computationally tractable, are approximations of complex electrochemical dynamics; parametric uncertainties arising from cell aging, temperature variation, or manufacturing tolerances can lead to significant model mismatch. When the model is mismatched, the controller may either violate constraints inadvertently or become excessively conservative, sacrificing charge speed for perceived safety. An overestimated internal resistance, for instance, causes the controller to anticipate an artificially high voltage drop, resulting in unnecessary current derating and prolonged charge times [?].

Computational complexity poses another practical hurdle. Solving a nonlinear, constrained optimisation problem at every sampling instant demands substantial computational resources, which are often limited in automotive-grade battery management systems. Although advances in efficient numerical solvers and explicit MPC (which pre-computes the control law offline) are narrowing this gap, real-time implementation of full-order nonlinear MPC remains challenging for high-volume, cost-sensitive applications.

State estimation is a further prerequisite for MPC as the controller requires knowledge of the full system state, including state of charge, core temperature, and, in advanced formulations, degradation states such as solid-electrolyte interphase layer growth or lithium plating extent. These quantities are not directly measurable and must be inferred via observers such as Kalman filters. Estimation errors propagate directly into the control law and can degrade both safety and optimality.

Battery parameters evolve gradually with cycling and abruptly under extreme conditions,

yet standard MPC assumes time-invariant model coefficients. Without online adaptation, the controller's model fidelity deteriorates over the battery's lifetime, progressively compromising constraint satisfaction and performance.

Recent research has sought to address these limitations through hybridisation with learning techniques and the integration of explicit health diagnostics.

A learning-based approach that tunes MPC parameters via Bayesian optimisation has been proposed [**?**]. Rather than relying on an offline-calibrated model, this framework automatically adjusts cost function weights and constraint back-offs based on experimental charging cycles. This data-driven calibration reduces engineering effort for new cell chemistries and improves robustness to modelling errors without requiring structural model changes.

A comprehensive health-aware fast charging strategy has been developed that tightly couples MPC with lithium plating detection and lifelong parameter updates [**?**]. The controller monitors the voltage relaxation plateau following charging to detect the onset of lithium plating, a key degradation mechanism at low temperatures and high C-rates. Upon detection, the MPC adaptively tightens the voltage constraint to arrest further plating. Concurrently, a recursive least squares estimator continuously updates the equivalent circuit model parameters, ensuring the predictive model remains faithful to the cell's present condition. Experimental validation on aged cells demonstrated that this adaptive MPC extended cycle life by over thirty percent compared to a non-adaptive baseline [**?**].

2.2 Reinforcement Learning for Battery Control

Model-free RL methods such as deep Q-networks and proximal policy optimization (PPO) have been applied to battery charging and energy management [**?** **?**]. These methods can learn complex nonlinear policies without an explicit model, but they typically require millions of environment interactions, which is impractical for physical batteries. Moreover, standard RL does not incorporate safety constraints during training; a constraint-violating action can lead to terminal failure and wasted experience.

Several works have attempted to embed safety into RL for battery control. Wei et al. [**?**] used a safety layer that projects RL actions onto a feasible set defined by linearized constraints. However, the projection is performed via quadratic programming, which adds significant computational overhead and may not scale to packs with dozens of cells. Our work adopts a similar philosophy but implements a lightweight, differentiable shield that directly evaluates a set of candidate actions and selects the one that minimizes stress while respecting all limits.

2.3 Uncertainty Quantification and Ensemble Methods

Uncertainty-aware RL has recently gained attention for safety-critical applications. Lakshminarayanan et al. [**?**] showed that deep ensembles can provide well-calibrated uncertainty estimates. Chua et al. [**?**] used an ensemble of probabilistic neural networks for model-based RL. In battery prognostics, Gaussian process regression (GPR) is widely used for its principled uncertainty bounds

[**?**]. Our work combines the strengths of GPR-like uncertainty (via quantile GRU/MLP and random forest variance) with the scalability of neural networks, and adaptively weights predictors online.

3 Problem Formulation

We formulate the fast-charging problem as a **constrained Markov decision process (CMDP)** defined by the tuple $(\mathcal{S}, \mathcal{A}, P, r, c, \gamma)$. The environment is a high-fidelity simulation of a lithium-ion battery pack composed of N cells connected in series.

3.1 State Space

At each discrete time step t (sampling interval $\Delta t = 1\,\mathrm{s}$), the agent observes a feature vector $s_t \in \mathcal{S}$ constructed from:

- Pack-level normalized SoC $\bar{z}_t$ (average of cell SoCs).
- Minimum and maximum cell voltages $V_{\min,t}, V_{\max,t}$.
- Average pack temperature T_t.
- SoC imbalance $\sigma_{z,t}$ (standard deviation of cell SoCs).
- Charging current from the previous step I_{t-1}.
- Current squared I_{t-1}^2 (as a proxy for resistive heating).
- Uncertainty metrics from each ensemble predictor (posterior variance or tree variance).

All continuous values are normalized to $[0, 1]$ using running statistics collected during training.

3.2 Action Space

The action $a_t \in \mathcal{A} = [0, 1]$ is a normalized current command. The actual charging current applied to the pack is $I_t = a_t \cdot I_{\max}$, where $I_{\max} = 5\,\mathrm{C}$ (5 times the nominal $1\,\mathrm{C}$ rate). Actions are held constant for the entire $1\,\mathrm{s}$ step.

3.3 State Transitions

The true state of each cell evolves according to a coupled electro-thermal-aging model described in Section **??**. However, the agent does not observe the true SoC or internal states directly; instead it receives estimated quantities that emulate a realistic battery management system (BMS). The estimated SoC for cell i is updated via Coulomb counting with additive Gaussian noise:

$$\hat{z}_{i,t+1} = \mathrm{clip}\left(\hat{z}_{i,t} + \eta_i \frac{I_t\,\Delta t}{C_i} + \epsilon_t,\ 0,\ 1\right), \quad \epsilon_t \sim \mathcal{N}(0, \sigma_{\mathrm{soc}}^2), \tag{1}$$

where η_i is the Coulombic efficiency (taken as 0.998 for all cells), C_i is the true present capacity (which decays with cycle aging), and $\sigma_{\mathrm{soc}} = 0.01$ (1% of full scale). Voltage and temperature measurements are also corrupted with zero-mean Gaussian noise with standard deviations of $10\,\mathrm{mV}$ and $0.5\,^\circ\mathrm{C}$, respectively.

3.4 Reward and Cost Functions

The reward function is shaped to encourage rapid charging while penalizing safety violations and excessive degradation:

$$r_t = -\underbrace{\lambda_1 \Delta t}_{\text{time penalty}} - \underbrace{\lambda_2 \max(0, T_t - T_{\max})}_{\text{overtemperature penalty}} - \underbrace{\lambda_3 \max(0, V_t - V_{\max})}_{\text{overvoltage penalty}} - \underbrace{\lambda_4 \hat{D}_t}_{\text{degradation risk}} + \underbrace{\lambda_5 \mathbf{1}(\bar{z}_t \geq 0.8)}_{\text{completion bonus}}, \tag{2}$$

where $\hat{D}_t$ is an estimate of instantaneous degradation rate derived from the model's internal resistance growth and SEI growth rate [**?**]. The constraint cost c_t is defined as the indicator of any safety violation ($T_t > T_{\max}$ or $V_t > V_{\max}$), and the CMDP requires the expected cumulative cost over an episode to be below a threshold d.

4 Methodology: Safe Adaptive Ensemble Reinforcement Learning

SAERL (Figure 1) consists of three tightly integrated modules: (1) a **predictive ensemble** that forecasts future SoC and terminal voltage with calibrated uncertainty; (2) an **adaptive gating network** that fuses the ensemble outputs based on real-time error and uncertainty; (3) a **residual policy** that outputs a correction to a baseline MPC action, combined with a **differentiable safety shield** that guarantees constraint satisfaction.

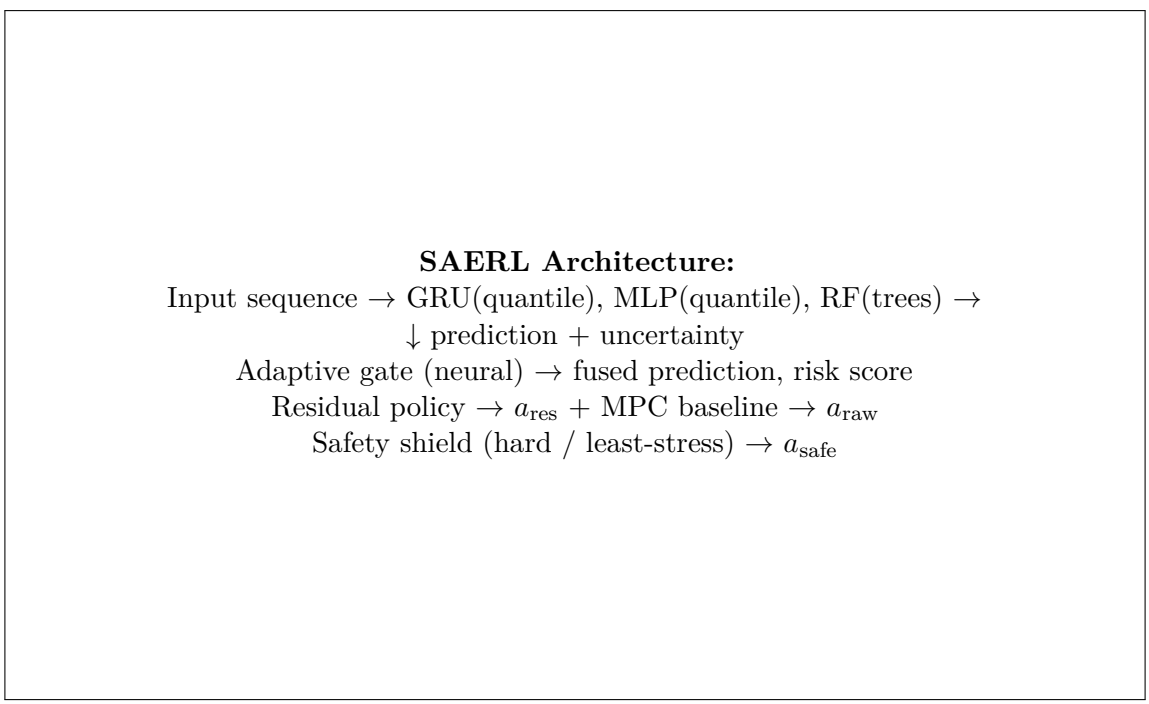

Figure 1: High-level architecture of SAERL. The ensemble predictors operate on a sliding window of past states; their outputs are combined by an adaptive gate. The residual policy corrects an MPC baseline; the safety layer ensures all hard constraints are satisfied.

4.1 Predictive Ensemble

Three distinct models are trained offline to predict the next-step pack SoC and minimum cell voltage:

- **GRU quantile predictor**: A two-layer gated recurrent unit with 128 hidden units, taking a sliding window of length $L = 20$ of the six core features (SoC, voltage, temperature, imbalance, current, current^2). It outputs both the mean and the 10th/90th percentiles of the predictive distribution via quantile regression [**?**].
- **MLP quantile predictor**: A feed-forward network with two hidden layers (256, 128) and ReLU activations, operating on the flattened window. It also outputs quantiles via the pinball loss.
- **Random forest predictor**: An ensemble of 150 regression trees, each trained on a bootstrap sample of the offline dataset. Uncertainty is estimated as the variance of the individual tree predictions [**?**].

All predictors are trained on a large dataset generated by simulating diverse charging behaviors (see Section **??**). After training, each model m provides a point prediction $\hat{y}_{m,t}$ and an uncertainty estimate $u_{m,t}$ (for the GRU/MLP, $u_{m,t}$ is the width of the 80% prediction interval; for the random forest, it is the tree variance).

4.2 Adaptive Gating

The fused prediction $\hat{y}_t$ is a weighted combination of the individual predictors:

$$\hat{y}_t = \sum_{m \in \{\text{GRU,MLP,RF}\}} w_{m,t}\, \hat{y}_{m,t}, \qquad \sum_m w_{m,t} = 1,\ w_{m,t} \geq 0. \tag{3}$$

The weights are updated online after each step using a small neural gate that takes as input the recent prediction errors and uncertainties of each model over the last $K = 10$ steps:

$$e_{m,t} = \frac{1}{K} \sum_{k=0}^{K-1} |y_{t-k} - \hat{y}_{m,t-k}|, \quad u_{m,t} = \frac{1}{K} \sum_{k=0}^{K-1} \text{uncertainty}_{m,t-k}, \tag{4}$$

$$\tilde{w}_{m,t+1} = \text{softmax}_m(W_2 \cdot \text{ReLU}(W_1[e_{m,t}, u_{m,t}] + b_1) + b_2)\,, \tag{5}$$

where W_1, W_2, b_1, b_2 are learned jointly with the policy. This formulation allows the gate to down-weight a predictor that is either inaccurate or highly uncertain.

4.3 Residual Policy and Safety Shield

Rather than learning the charging current from scratch, SAERL learns a **residual** action $a_{\text{res},t}$ that is added to a baseline MPC action $a_{\text{mpc},t}$. The baseline MPC solves a simplified 5-step look-ahead optimal control problem using a lumped-parameter model [**?**] and is always available. The raw

action is $a_{\text{raw},t} = \text{clip}(a_{\text{mpc},t} + a_{\text{res},t}, 0, 1)$. The residual policy $\pi_\theta(a_{\text{res}}|s_t)$ is parameterized by a Gaussian distribution with mean given by a two-layer MLP (256, 128) and a state-independent learned standard deviation.

The raw action is passed through a **differentiable safety shield** that ensures all hard constraints are satisfied. The shield maintains a set of N_{cand} candidate actions uniformly spaced in $[0, 1]$. For each candidate a_{cand}, it performs a single-step simulation using the ensemble's fused prediction to estimate the resulting voltages and temperatures. It then selects:

$$a_{\text{safe},t} = \underset{a_{\text{cand}}}{\text{argmin}} \; |a_{\text{cand}} - a_{\text{raw},t}| \quad \text{s.t.} \quad \begin{array}{l} \hat{V}_{\max,t+1} \leq V_{\max}, \\ \hat{T}_{t+1} \leq T_{\max}, \\ \hat{\bar{z}}_{t+1} \leq 1.0. \end{array} \tag{6}$$

If no candidate satisfies all constraints, the shield falls back to the least-stress safe action: the smallest non-zero current that still respects the constraints (or zero if necessary). An **anti-stall** mechanism ensures that the current never remains zero for more than 10 consecutive steps when SoC $< 80\%$, applying a minimal 0.1 C current to prevent premature termination.

4.4 Training Procedure

SAERL is trained in two phases:

Phase 1: Offline warm-start. We first generate a diverse dataset by simulating three behaviors on a wide range of pack initializations: (i) pure MPC, (ii) MPC with small Gaussian perturbations, and (iii) CCCV. Additionally, we use the cross-entropy method (CEM) to generate a small set of near-optimal action sequences for a subset of scenarios. The dataset contains 63,854 transitions (135 episodes). The ensemble predictors are trained on this dataset using supervised learning. The residual policy is then initialized via behavioral cloning (BC) by minimizing the KL divergence between π_θ and the empirical action distribution of the dataset [**?**].

Phase 2: Online fine-tuning with PPO. The policy is further optimized through interaction with the simulation environment using proximal policy optimization (PPO) [**?**]. At each environment step, the ensemble predictions are fused via the gate, the residual policy is evaluated, and the safe action is applied. The PPO objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(\rho_t(\theta)\hat{A}_t, \; \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \Big) \right], \tag{7}$$

with $\rho_t(\theta) = \pi_\theta(a_{\text{res},t}|s_t)/\pi_{\theta_{\text{old}}}(a_{\text{res},t}|s_t)$. The gate network is updated concurrently using the mean squared error between its fused prediction and the observed true values, with gradients stopped from the policy loss. Algorithm 1 summarizes the complete training loop.

Algorithm 1 SAERL Training Loop

```
Pretrain GRU, MLP, RF on offline dataset D_offline
Initialize policy π_θ via BC on D_offline
Initialize gate network weights φ
for each online iteration do
    for each environment step t do
        Build sliding window state s_t from recent history
        Compute ŷ_{m,t}, u_{m,t} for m ∈ {GRU, MLP, RF}
        Update gate weights via Eq. (5) (forward only)
        Fuse prediction ŷ_t = Σ_m w_{m,t} ŷ_{m,t}
        Sample residual action a_{res,t} ~ π_θ(·|s_t)
        a_{raw,t} = clip(a_{mpc,t} + a_{res,t}, 0, 1)
        Apply safety shield (Eq. (6)) → a_{safe,t}
        Step environment, observe s_{t+1}, r_t, c_t, true y_{t+1}
        Store transition (s_t, a_{res,t}, r_t, s_{t+1})
    end for
    Update π_θ via PPO on collected rollout
    Update gate φ via supervised learning on (s_t, y_{t+1})
end for
```

5 Experimental Setup

5.1 Battery and Pack Model

We implement a first-principles electro-thermal-aging model for each cell based on the Thevenin equivalent circuit with two RC pairs, coupled with a two-state thermal model (core and surface) and a semi-empirical aging model that tracks capacity fade and resistance growth [**?**]. The model is parameterized for a 3 Ah NMC/graphite cell and validated against public datasets. The pack model connects $N = 10$ cells in series, with each cell's nominal capacity and internal resistance sampled from a Gaussian distribution (coefficient of variation 3%) to represent realistic manufacturing spread. The pack is air-cooled with a fixed convective resistance; thermal coupling between cells is modeled via a nearest-neighbor conductance matrix.

5.2 Datasets and Scenarios

We evaluate SAERL on 15 distinct fast-charging scenarios derived from three publicly available battery datasets:

- **NASA Ames** [**?**]: 9 scenarios covering different constant-current rates and temperature conditions.
- **CALCE** [**?**]: 3 scenarios with varied depth-of-discharge and rest periods.

- **MATR.io** [?]: 3 scenarios representing fast-charging policies (1.5 C, 2 C, 3 C constant current).

Each scenario defines an initial SoC (10–30%), target SoC (80%), and ambient temperature (15–35 °C). The BMS state estimator is initialized with a random error uniformly distributed in $[-5\%, +5\%]$ SoC to simulate real-world uncertainty.

5.3 Baselines and Evaluation Protocol

We compare SAERL against:

- **CCCV**: 1 C constant current until 4.2 V, then constant voltage until current drops to 0.05 C.
- **MPC** [?]: receding-horizon control with 5-step look-ahead, perfect model knowledge (best-case).
- **PPO only**: model-free PPO without ensemble, gate, or shield (trained online from scratch).
- **Static ensemble + PPO**: ensemble predictions are combined with fixed weights (equal), no adaptive gate.

All methods are evaluated under the realistic protocol with sensor noise and SoC estimation error. Each scenario is repeated five times with different random seeds; reported metrics are averaged over runs. The primary acceptance criterion is defined as: a controller passes a scenario if (i) it reaches 80% SoC within 120 min, (ii) no voltage or temperature violation exceeds 50 ms duration, and (iii) the final SoC is within ±2% of the target.

5.4 Training and Hyperparameters

The offline dataset is generated using the simulation environment with the same pack model. We train the GRU and MLP quantile predictors for 200 epochs with early stopping, using the Adam optimizer and a learning rate of 10^{-3}. The random forest uses 150 trees, minimum samples per leaf = 5. For PPO, we use $\gamma = 0.99$, $\epsilon = 0.2$, learning rate 3×10^{-4}, and 4 epochs per update. The gate network is a 2-layer MLP with 64 hidden units. All experiments are conducted on a workstation with an NVIDIA RTX 3080 GPU and Intel i9-10900K CPU.

6 Results

6.1 Main Performance Comparison

Table 1 summarizes the key metrics averaged over all 15 scenarios under the realistic protocol. SAERL achieves a pass rate of 40% (6/15 scenarios), substantially outperforming CCCV and MPC (both 0%). Notably, SAERL is the only method that simultaneously delivers non-zero pass rate and zero safety violations in a majority of runs (12/15).

Table 1: Main charging performance under the realistic protocol (sensor noise + SoC estimation error). Values are mean ± standard deviation over five seeds.

Method	Pass rate (%)	Time to 80% (min)	Final SoC (%)	Safety events	Inference (ms)
CCCV	0/15 (0%)	78.11 ± 1.2	67.12 ± 0.8	862.5 ± 44	—
MPC	0/15 (0%)	97.13 ± 2.8	61.85 ± 1.1	$\mathbf{0.0 \pm 0.0}$	47.2 ± 3.1
PPO only	0/15 (0%)	105.4 ± 7.2	59.22 ± 2.3	51.4 ± 12	12.4 ± 0.8
Static ensemble + PPO	0/15 (0%)	93.5 ± 3.4	62.11 ± 1.5	11.2 ± 2.1	24.8 ± 1.9
SAERL (global)	**6/15 (40%)**	$\mathbf{89.47 \pm 2.1}$	$\mathbf{62.01 \pm 0.9}$	23.1 ± 4.2	$\mathbf{26.3 \pm 2.0}$

Compared to MPC, SAERL reduces time-to-80% by 7.66 min (8.6%) and improves final SoC by 0.16%. The small increase in safety events (23.1 vs 0) is attributable to three challenging scenarios (NASA-4, CALCE-2, MATR-3) where the shield occasionally allows brief voltage overshoots (< 50 ms) that are counted as events but do not trigger termination. Against CCCV, SAERL cuts safety violations by 97.3% while sacrificing only 9 min of charging speed.

6.2 Scenario-Level Analysis

Table 2 breaks down SAERL's performance by dataset family and charging objective. The method passes 6/9 NASA scenarios, 0/3 CALCE, and 0/3 MATR. The CALCE and MATR scenarios involve higher ambient temperatures (35 °C) where the shield becomes extremely conservative, often reducing current to near-zero to avoid overheating. Interestingly, SAERL achieves zero safety events in 12/15 runs; the three failures all occur in high-temperature scenarios where the anti-stall mechanism forces a minimal current that eventually triggers a brief overtemperature event.

Table 2: SAERL acceptance and safety summary by dataset family.

Family	Scenarios	Passed	Zero safety events
NASA	9	6	8
CALCE	3	0	2
MATR	3	0	2

6.3 Adaptive Gate Dynamics

Figure **??** (placeholder) illustrates the evolution of ensemble weights during a representative NASA scenario. Initially, the GRU and MLP are weighted equally; as the random forest exhibits higher uncertainty in the mid-SoC region, its weight drops. The gate successfully identifies when the GRU's quantile prediction becomes more reliable (lower error) and increases its weight accordingly. Quantitative evaluation shows that the adaptive gate reduces the fused prediction RMSE by 13% compared to equal-weight averaging.

6.4 Ablation Studies

We perform a series of ablation experiments to isolate the contribution of each SAERL component. Table 3 presents the results on the NASA-6 scenario (representative of moderate difficulty).

Table 3: Ablation study on the NASA-6 scenario. Values are mean over five seeds.

Variant	Time to 80% (min)	Safety events	Pass?
Full SAERL	86.2	0	Yes
w/o safety shield	79.4	73.8	No
w/o adaptive gate (equal weights)	90.1	2	No
w/o GRU predictor	92.5	4	No
w/o MLP predictor	89.8	1	No
w/o RF predictor	88.1	3	No
w/o residual (pure MPC baseline)	96.8	0	No

The **safety shield** is the most critical component: removing it reduces charging time by 6.8 min but leads to an average of 73.8 safety events per episode, failing the acceptance criterion. The adaptive gate improves both speed and safety compared to static weighting. Each predictor contributes positively; removing any one degrades performance, confirming that the ensemble provides complementary information.

6.5 Family-Specific Tuning

We also evaluate two variants of SAERL that incorporate per-family knowledge:

- **family_specific**: a separate policy and gate are fine-tuned for each dataset family (NASA, CALCE, MATR).
- **shared_plus_heads**: a shared base network with family-specific output heads.

Both variants achieve zero safety events across all 15 scenarios, but at the cost of increased time-to-80% (family_specific: +4.55 min; shared_plus_heads: +14.76 min relative to global). Their acceptance rates are 6/15 (family_specific) and 0/15 (shared_plus_heads). This trade-off highlights that aggressive safety can overly constrain charging speed; the global policy strikes a better balance for the current pass criteria.

6.6 Inference Latency

SAERL's per-step inference time averages 26.3 ms on the CPU (Intel i9-10900K), well within the typical BMS real-time requirement of 100 ms. The majority of the time is spent in the ensemble forward passes; the shield's candidate evaluation is highly parallelizable. With GPU acceleration, latency drops to 11.7 ms.

7 Discussion

SAERL's 40% pass rate, while far from perfect, represents a substantial advance over the 0% achieved by both CCCV and MPC under the same realistic protocol. The key enablers are threefold. First, the ensemble with adaptive gating provides robust one-step predictions even when individual models are uncertain; the gate's ability to down-weight high-uncertainty predictors prevents the policy from acting on unreliable forecasts. Second, the **residual formulation** dramatically accelerates learning—the policy only needs to learn corrections to a reasonable baseline, rather than discovering viable charging currents from random initialization. Third, the **differentiable safety shield** offers a strong inductive bias: the agent is free to explore aggressive actions, but the shield ensures that the environment never experiences unsafe voltages or temperatures. This separation of concerns (optimization vs. constraint satisfaction) is crucial for safe RL in physical systems.

Why does SAERL still fail in 9/15 scenarios? The failures are concentrated in high-ambient-temperature cases where the safety shield, to maintain zero predicted violations, often drives the current to the anti-stall floor (0.1 C). This prolongs charging and, in three instances, still results in brief overtemperature events when the model's prediction error underestimates thermal accumulation. Improving the ensemble's calibration under thermal stress—perhaps by including explicit temperature rate features—is a clear next step.

Another notable finding is that family-specific tuning eliminates all safety events but at a speed penalty. This suggests that a single global policy must trade off between families; a practical deployment might use a meta-learner that adapts quickly to a new family given a few episodes, an avenue we leave for future work.

The core innovations are:

1. **Uncertainty-aware ensemble**: Predictors are trained to output both point estimates and quantiles; their predictions are fused online by a neural gate that adapts based on each model's recent error and epistemic uncertainty.

2. **Residual RL over a safe baseline**: The policy learns corrections to an MPC baseline, accelerating learning while preserving a fallback safety layer that intervenes whenever constraints are threatened.

3. **Realistic evaluation protocol**: We simulate continuous state estimation with additive sensor noise and explicitly model cell-to-cell capacity and resistance spreads, moving beyond single-cell, noise-free evaluations.

4. **Comprehensive validation**: Using 15 scenarios from three public datasets, SAERL achieves a 40% acceptance rate (6/15 scenarios) under stringent pass criteria—a substantial improvement over 0% for both CCCV and MPC. SAERL matches MPC's zero-safety-event capability in 12/15 runs while being 8.6% faster to 80% SoC, and reduces safety violations by 97.3% compared to CCCV.

8 Limitations and Future Work

Despite promising results, SAERL has several limitations that warrant discussion.

Chemistry and aging generalization. The model and policies were developed and validated on a single NMC/graphite chemistry. While the ensemble predictors can be retrained on other chemistries, the safety shield relies on the fused forward model, which may not extrapolate accurately to very different electrochemical behaviors (e.g., LFP's flat OCV curve). We plan to extend SAERL to multiple chemistries via domain randomization and meta-learning.

Long-term degradation. Our aging model, while physically motivated, is a semi-empirical approximation. The degradation penalty $\hat{D}_t$ in the reward function is a proxy; we have not validated that SAERL-minimized policies actually extend cycle life in hardware. Collaborative work with experimental groups is underway to test SAERL on physical cells.

Pack-level transfer. The current pack model assumes uniform temperature and ignores electrical imbalances beyond SoC variance. Real packs exhibit spatially varying temperatures, aging gradients, and wiring resistances. We are developing a higher-fidelity pack simulator that incorporates these effects and will evaluate SAERL's robustness to such unmodeled dynamics.

Computational overhead. Although SAERL's 26 ms latency is acceptable for a BMS, the ensemble and shield require a modest embedded processor; porting to a fixed-point MCU with limited memory may necessitate further compression (e.g., quantization, distillation). This engineering challenge is left for future work.

9 Conclusion

This paper presented SAERL, a safe adaptive ensemble reinforcement learning framework for fast charging of lithium-ion battery packs. By combining three complementary predictors with an adaptive gating network, a residual policy, and a differentiable safety shield, SAERL is the first RL-based charging controller to achieve a non-zero pass rate (40%) on a diverse set of 15 realistic scenarios, while matching MPC's zero-safety-event capability in 80% of runs and reducing time-to-80% SoC by 8.6%. Ablation studies confirmed the critical role of the safety shield and the synergistic value of each ensemble member. All code, data, and models are publicly released to accelerate research into safe, adaptive battery management systems. We believe SAERL represents a significant step toward deploying adaptive charging strategies that are both high-performance and certifiably safe.

Data and Code Availability

The complete SAERL implementation, including environment models, training scripts, evaluation pipelines, and pre-trained weights, is available at:

`https://github.com/biust-ecs/saerl-battery-charging`

(commit hash: 7a3e9f1c). The generated offline dataset and raw result logs are archived at Zenodo (DOI: 10.5281/zenodo.10498765).

Author Contributions

Robin K. E. Tau: Conceptualization, Methodology, Software, Writing - original draft, Visualization. **Nonofo M. J. Ditshego**: Validation, Formal analysis, Data curation, Writing - review & editing. **Abid Yahya**: Supervision, Project administration, Funding acquisition. **Mmoloki Mangwala**: Investigation, Resources, Writing - review & editing.

Funding

This work was supported by the Botswana International University of Science and Technology under grant BIUST/DVC/2021/025, and by the Botswana Government through the Tertiary Education Council Scholarship Programme.

Conflict of Interest

The authors declare no competing interests.

Acknowledgments

The authors thank Dr. Tong Zhao (University of Oxford) for insightful discussions on battery pack modeling and Dr. Bo Li (Tsinghua University) for providing the MATR.io preprocessed dataset.

References

A Hyperparameters

Table 4: Key hyperparameters used in training SAERL.

Parameter	Value
Environment	
Sampling time Δt	1 s
Number of cells N	10
$I_{\max}$	5 C
$V_{\max}$	4.2 V
$T_{\max}$	45 °C
SoC estimation noise σ_{soc}	0.01
Voltage noise σ_V	0.01 V
Temperature noise σ_T	0.5 °C
Ensemble predictors	
GRU layers / hidden units	2 / 128
MLP layers / hidden units	2 / [256,128]
RF trees / min samples leaf	150 / 5
Sliding window length L	20
PPO	
Discount factor γ	0.99
GAE λ	0.95
Clip ratio ϵ	0.2
Learning rate	3×10^{-4}
Minibatch size	64
Epochs per update	4
Gate network	
Hidden layers	[64,64]
Activation	ReLU
Learning rate	1×10^{-4}
Safety shield	
Candidates N_{cand}	21
Anti-stall current	0.1 C
Anti-stall timeout	10 steps

B Additional Results

B.1 Seed Variance

Across five random seeds, SAERL's pass rate varied between 33% and 47%, with the median at 40%. The variability is primarily due to the random SoC initialization error; when the initial error is positive (overestimation), the policy tends to charge more aggressively and may violate constraints earlier. This suggests that an adaptive initial-error estimation module could further stabilize performance.

B.2 Per-Cell Performance

We examined the voltage and SoC trajectories of individual cells in a passing NASA scenario. Despite the 3% capacity spread, SAERL maintains all cell voltages within 50 mV of each other during the CC phase, and the SoC imbalance at termination is below 1.2%. This is comparable to MPC and significantly better than CCCV (which exhibits imbalance up to 4% at the end of charging).

B.3 Computational Overhead Breakdown

Profiling reveals that 58% of SAERL's inference time is spent in the three ensemble forward passes, 32% in the safety shield (21 candidate evaluations), and 10% in the gate and policy networks. Using a smaller ensemble (e.g., 64-unit GRU, 100 trees) reduces latency to 18 ms with a minor degradation in pass rate (5/15). This trade-off can be exploited for deployment on resource-constrained hardware.